Capitolo 9

Design pattern comportamentali e MVC

9.1 Algoritmi, comunicazione e distribuzione delle responsabilità

I *design pattern comportamentali* organizzano algoritmi, comunicazione e distribuzione delle responsabilità fra oggetti. Specificano chi esegue un comportamento, quando lo esegue e come gli oggetti cooperano attraverso contratti, evitando che ogni partecipante conosca i dettagli di tutti gli altri.

MVC (*Model–View–Controller*) affronta un problema su scala diversa: è un pattern architetturale che organizza un'applicazione interattiva in dati e logica, presentazione e gestione dell'input. Può usare collaborazioni comportamentali, in particolare Observer, all'interno di questa separazione di responsabilità.

Un *algoritmo* è una sequenza ordinata di passi per ottenere un risultato; *incapsularlo* significa rappresentarlo dietro un'operazione o un oggetto, così che chi lo usa non dipenda dai dettagli. I pattern del capitolo risolvono famiglie differenti di problemi:

Template Method fissa lo scheletro di un algoritmo nella superclasse e rinvia passi variabili alle sottoclassi.

Strategy separa algoritmi alternativi intercambiabili dal contesto che li usa.

State sostituisce i rami condizionali dipendenti dallo stato con oggetti che rappresentano gli stati.

Observer realizza una dipendenza uno-a-molti: un cambiamento di un soggetto notifica automaticamente gli osservatori registrati.

Mediator concentra in un mediatore la coordinazione fra molti oggetti collaboranti.

Chain of Responsibility fa percorrere una richiesta a una catena di possibili gestori, disaccoppiando mandante e ricevente.

Command rende una richiesta un oggetto, quindi la rende memorizzabile, accodabile, componibile e, se progettato a questo fine, annullabile.

Iterator separa il percorso di una collezione dalla sua rappresentazione interna.

Null Object sostituisce un riferimento assente con un oggetto conforme al contratto che esegue un comportamento neutro.

La scelta non dipende dalla somiglianza del diagramma UML. In particolare State e Strategy hanno entrambi un Context che delega a un'interfaccia, ma il primo rappresenta l'evoluzione interna di una macchina a stati; il secondo sceglie un algoritmo alternativo. Per scegliere il pattern occorre quindi partire dal suo scopo e dalla causa del cambiamento.

9.2 Template Method: scheletro fisso e passi specializzati

Più classi possono eseguire gli stessi passi nello stesso ordine, differendo soltanto nei dettagli di alcuni di essi. Copiare l'intero algoritmo in ciascuna classe costringerebbe a ripetere le modifiche comuni e potrebbe renderle incoerenti.

Template Method raccoglie lo scheletro dell'algoritmo in un'operazione della superclasse e affida alle sottoclassi la definizione di alcuni passi. Le specializzazioni cambiano quei dettagli senza cambiare l'ordine complessivo.

`AbstractClass` dichiara le *operazioni primitive*, cioè i passi indispensabili che le sottoclassi devono implementare, e definisce il template method. `ConcreteClass` implementa tali primitive.

Un **hook** è invece un'operazione di aggancio con implementazione predefinita, spesso vuota: la sottoclasse può ridefinirla per personalizzare un punto dell'algoritmo ma non è obbligata. Il template può anche invocare operazioni concrete della superclasse, operazioni sovrascritte della sottoclasse, operazioni di classi client e metodi factory. Un *metodo factory* è qui un passo che crea un oggetto il cui tipo concreto è deciso dalla sottoclasse.

Il vantaggio è il riuso della parte invariabile, particolarmente utile nelle librerie e nei framework. Un *framework* è un'infrastruttura che chiama il codice applicativo nei punti previsti: qui vale il principio di Hollywood, "non chiamarci, noi ti chiameremo", perché la superclasse controlla il flusso e invoca i passi della sottoclasse.

Il prezzo è il legame di ereditarietà: la classe concreta dell'oggetto fissa la variante, che non può essere sostituita semplicemente cambiando un riferimento durante l'esecuzione.

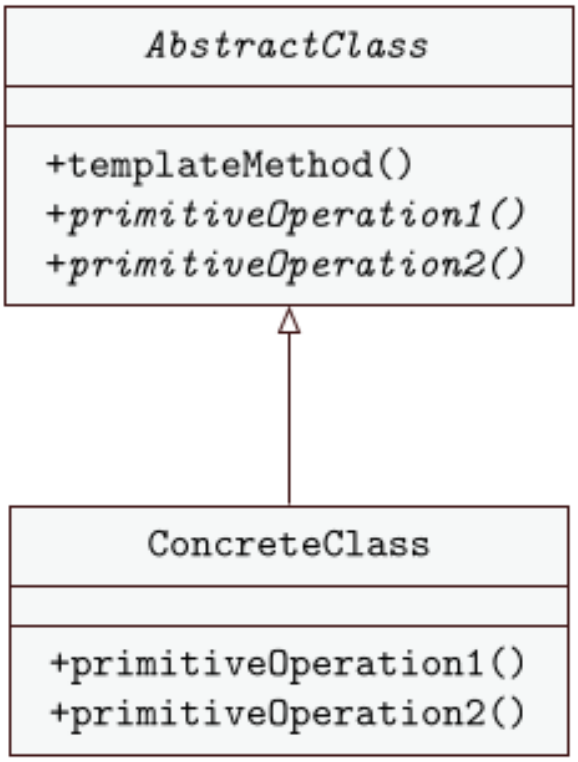


Figura 9.1: AbstractClass definisce il template e le primitive; ConcreteClass implementa i passi.

Un factory method usato come passo del template può essere una primitiva astratta o avere un prodotto predefinito quando esiste una scelta sensata. Il tipo di ritorno Product non garantisce che sia possibile istanziarlo direttamente: può essere astratto o un'interfaccia.

Rendere il template `final` è una possibile decisione per proteggere la sequenza dai cambiamenti delle sottoclassi; la keyword non è necessaria ogni volta che si applica il pattern.

9.2.1 Esempio: apertura di un documento

Nell'esempio Template Method, `Application.openDocument()` determina la sequenza per aprire e leggere un documento. `MyApplication` specializza la creazione e la verifica del documento; `MyDocument` realizza i dettagli della lettura. Il diagramma separa quindi la sequenza comune dai passi affidati alle classi concrete.

Il successivo frammento con AbstractClass mostra lo stesso meccanismo in forma minima: il template invoca prima primitiveOperation1 e poi primitiveOperation2; ConcreteClass ne fornisce le implementazioni. Il ricevente rimane l'oggetto concreto e il dispatch seleziona i suoi passi specializzati, mentre l'ordine resta definito nella superclasse.

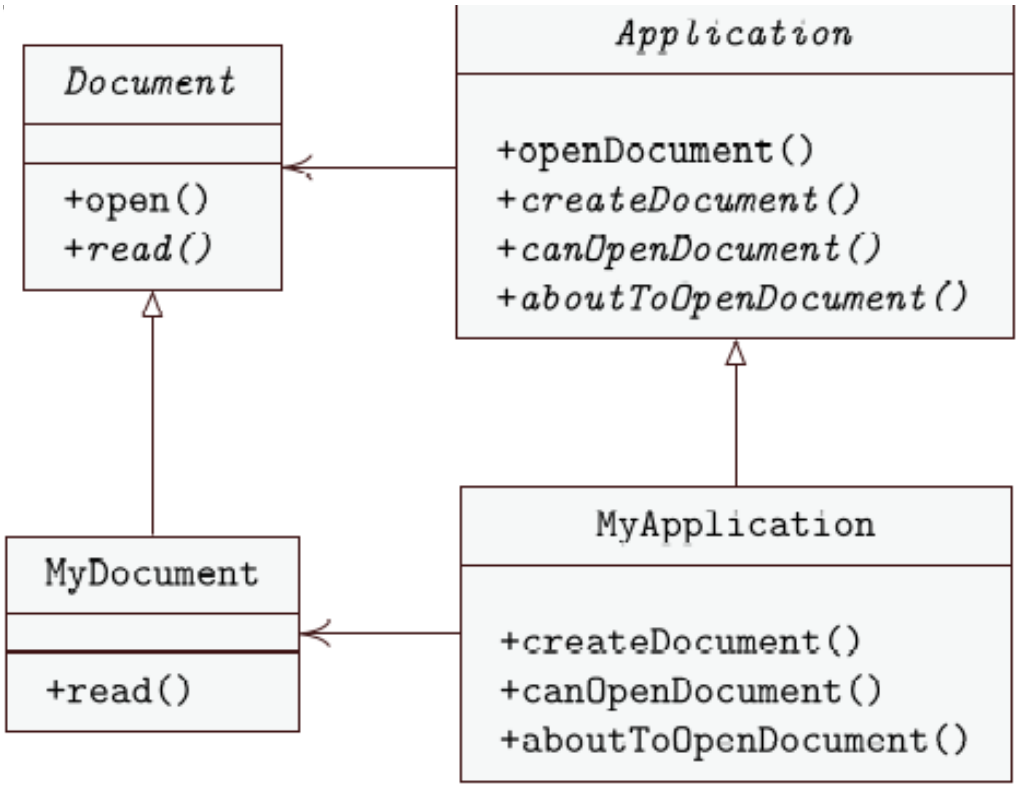


Figura 9.2: Application e Document mantengono la struttura comune; MyApplication e MyDocument specializzano i passi.

```
public abstract class AbstractClass {
    public void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
    }
    protected abstract void primitiveOperation1();
    protected abstract void primitiveOperation2();
}
```

```
public class ConcreteClass extends AbstractClass {
    @Override
    protected void primitiveOperation1() {
        // codice per l'operazione
    }
    @Override
    protected void primitiveOperation2() {
    }
}
```

Figura 9.3: Implementazione del template che invoca primitiveOperation1 e primitiveOperation2 ridefinite nella sottoclasse.

9.3 State: comportamento dipendente dallo stato corrente

Il problema tipico nasce quando molti metodi decidono il comportamento leggendo lo stesso *type code*: una variabile enumerativa o numerica che rappresenta lo stato. Ogni metodo ripete `if/else` o `switch`; aggiungere uno stato richiede di riesaminare tutte queste decisioni. La duplicazione viola DRY (*Don't Repeat Yourself*, non ripetere la stessa conoscenza).

State permette di cambiare il comportamento dell'oggetto quando cambia il suo stato interno. Dal punto di vista del client sembra che l'oggetto abbia cambiato classe: il comportamento richiesto viene affidato a un diverso oggetto di stato, secondo la collaborazione descritta sotto.

Il client usa il `Context`, che mantiene un riferimento persistente allo `State` corrente. Quando riceve una richiesta, per esempio `request()`, il contesto la delega a `currentState.handle(...)`.

Il contratto `State` dichiara le operazioni comuni; ciascun `ConcreteState` fornisce il comportamento del proprio stato. Il Context può passare sé stesso a `handle`: il destinatario può così leggere i dati del contesto o invocare `setState`. Il riferimento conservato dal contesto determina quale implementazione riceverà la richiesta successiva.

Collocazione delle transizioni

Le transizioni possono essere centralizzate nel Context: in questo modo è più facile leggerle nel loro insieme. In alternativa possono essere distribuite nei ConcreteState, ciascuno dei quali conosce il proprio successore. In entrambi i casi la transizione sostituisce un riferimento a State, non confronta un type code.

State raccoglie comportamenti e transizioni nelle classi di stato e permette di aggiungerne di nuove. Il costo è l'aumento del numero di classi. Il refactoring associato è *Replace Type Code with State*, caso di *Replace Conditional with Polymorphism*: ogni ramo dello `switch` diventa un'implementazione polimorfica. Nell'esempio del tornello, `LockedState` sa che l'inserimento di un gettone porta a `Unlocked`; affidargli la transizione evita che Context ricostruisca un'altra catena di condizioni per riconoscere lo stato corrente.

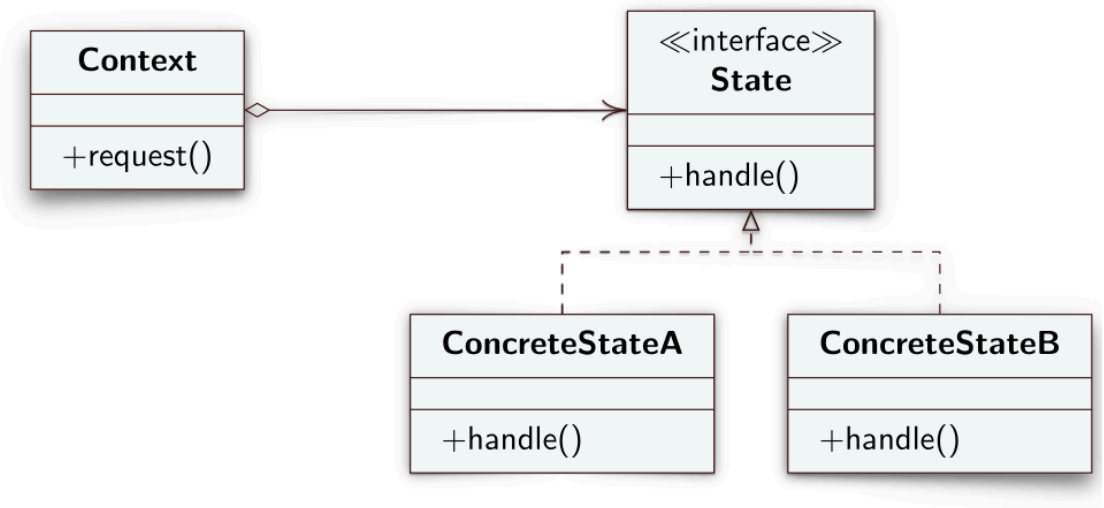


Figura 9.4: Context delega allo State corrente e i ConcreteState forniscono i comportamenti.

Un ConcreteState può cambiare il contesto ricevuto come parametro oppure restituire il nuovo State affinché il Context lo assegni. Il seguente schema rende esplicita la seconda possibilità:

```
interface Stato {
    Stato evento();
}
final class Contesto {
    private Stato corrente;
    Contesto(Stato iniziale) {
        corrente = java.util.Objects.requireNonNull(iniziale);
    }
    void evento() {
        corrente = corrente.evento();
    }
}
```

Le implementazioni devono restituire uno stato valido, eventualmente `this` quando non cambia. La logica delle transizioni può stare nel Context o nei ConcreteState; non è obbligatorio collocare la logica delle transizioni in entrambi nello stesso tempo. Per condividere eventualmente le istanze di stato, queste non devono contenere dati mutabili specifici di un singolo contesto.

9.3.1 Transizioni e concatenamento delle chiamate

Un'operazione di stato può restituire il successore da installare nel contesto. Se i tipi restituiti mettono a disposizione le operazioni successive, le chiamate possono essere concatenate in una forma *fluent*. Si consideri uno stato concreto `Bloccato`, la cui operazione `inTesta(s)` riceve l'argomento `s` e restituisce uno stato. L'espressione

```
return new Bloccato().inTesta(s);
```

equivale ai passi:

```
var b = new Bloccato();
var successivo = b.inTesta(s);
return successivo;
```

Il risultato è quello di `inTesta(s)`, non necessariamente l'istanza appena costruita. Un ulteriore `.paga()` è possibile soltanto se il tipo restituito espone tale operazione: il risultato della catena è quello dell'ultima chiamata. La notazione fluente non impone quindi che tutti i metodi restituiscano `this`.

9.3.2 Esempio: modalità di scrittura di un libro

`Libro` conserva un collaboratore `Display` che realizza `scrivi`; `Colonna` e `SingolaParola` ne sono implementazioni alternative. I metodi `modeColonna()` e `modeParola()` sostituiscono Display, evitando di ripetere `switch(mode)` nelle operazioni del libro. La struttura realizza State quando Display rappresenta lo stato del contesto e le sue evoluzioni; se rappresenta soltanto un algoritmo di formattazione scelto dal client, l'intento è invece quello di Strategy. Il criterio decisivo è il ruolo svolto dal collaboratore, non il solo diagramma delle classi.

Dal selettore numerico agli oggetti Display

La versione condizionale `LibroPrimaDiState` conserva `mode=2`; `mostra` esegue uno `switch`, con caso 1 per la singola parola e caso 2 per la colonna, mentre `setMode(int)` sostituisce il codice. Il refactoring trasferisce i due corpi in classi separate:

```
import java.util.*;

interface Display { void scrivi(List<String> testo); }
class Libro {
    private final String testo =
        "Darwin's _Origin of Species_ persuaded the world that the " +
        "difference between different species of animals and plants " +
        "is not the fixed immutable difference that it appears to be.";
    private final List<String> lista =
```

```
        Arrays.asList(testo.split("[\\s+]+"));
    private Display mode = new Colonna();
    public void mostra() { mode.scrivi(lista); }
    public void modeColonna() { mode = new Colonna(); }
    public void modeParola() { mode = new SingolaParola(); }
}
class Colonna implements Display {
    private final int numCar = 38, numRighe = 12;
    public void scrivi(List<String> testo) {
        int riga = 0, col = 0;
        for (String p : testo) {
            if (col + p.length() > numCar) {
                System.out.println(); riga++; col = 0;
            }
            if (riga == numRighe) break;
            System.out.print(p + " ");
            col += p.length() + 1;
        }
    }
}
class SingolaParola implements Display {
    public void scrivi(List<String> testo) {
        int maxLung = 0;
        for (String p : testo) maxLung = Math.max(maxLung, p.length());
        System.out.println();
        for (String p : testo) {
            int sinistra = (maxLung - p.length()) / 2;
            int destra = maxLung - p.length() - sinistra;
            System.out.print(" ".repeat(sinistra) + p + " ".repeat(destra));
            try { Thread.sleep(300); }
            catch (InterruptedException e) {
                Thread.currentThread().interrupt(); break;
            }
            System.out.print("\b".repeat(maxLung));
        }
        System.out.println();
    }
}
```

Il client crea Libro, chiama `mostra`, poi `modeParola` e di nuovo `mostra`. Il contesto conserva lo stesso testo ma delega al nuovo collaboratore.

La regex divide su sequenze di spazi bianchi o del carattere `+`; per dividere soltanto sugli spazi bianchi basta `"\\s+"`.

`Colonna` va a capo quando la parola supererebbe 38 caratteri e interrompe dopo 12 righe; il semplice algoritmo non spezza parole più lunghe della larghezza e conta anche lo spazio stampato dopo ciascuna parola.

`SingolaParola` calcola la larghezza massima, centra ciascuna parola, attende 300 millisecondi e tenta di riportare indietro il cursore. La divisione intera assegna alla destra l'eventuale spazio dispari: incrementare lo spazio destro soltanto per parole di lunghezza dispari sarebbe sufficiente solo con larghezza massima pari.

Il carattere backspace non garantisce cancellazione visiva in ogni terminale; l'effetto dipende

dal dispositivo di uscita. Una modalità a due colonne sarebbe un'ulteriore implementazione di Display, con un proprio algoritmo di impaginazione.

Nella variante di State in cui il Context decide la transizione, un'operazione dello stato può restituire un booleano che segnala esito o condizione raggiunta; il Context lo interpreta e aggiorna lo stato. È un'alternativa alla restituzione diretta dello State successivo, non una regola secondo cui ogni operazione debba cambiare stato.

9.4 Strategy: algoritmi alternativi intercambiabili

Strategy incapsula una famiglia di algoritmi alternativi. Il contratto `Strategy` descrive l'algoritmo richiesto; una `ConcreteStrategy` ne realizza una variante. Il `Context` conserva il collaboratore scelto e gli delega l'esecuzione.

Si usa quando classi correlate differiscono soprattutto per un comportamento, quando il Context deve poter essere configurato con una fra più varianti, oppure quando le varianti dello stesso algoritmo offrono compromessi diversi nell'uso delle risorse. Il cambiamento da isolare è quindi la scelta dell'algoritmo.

A differenza di Template Method, la variazione usa composizione e può essere sostituita a runtime.

A differenza di State, normalmente il client o il Context sceglie la strategia dall'esterno, in base alla configurazione o alle necessità; le strategie non rappresentano una sequenza di stati e non si passano il controllo l'una all'altra. La struttura può coincidere, ma l'intento resta il criterio decisivo.

Il vantaggio è aggiungere algoritmi senza modificare Context ed evitare condizionali; il costo è che il client deve conoscere le differenze rilevanti fra le strategie e che un contratto troppo ampio può trasferire parametri inutili alle varianti più semplici.

Precisazione

State e Strategy possono entrambi cambiare collaboratore a runtime. La distinzione è l'intento: evoluzione dello stato del contesto rispetto a scelta di un algoritmo. ConcreteStrategy normalmente non conosce le strategie alternative; ConcreteState può conoscere gli stati successivi e guidare la transizione. Questa distinzione riguarda il modo in cui si assegnano le responsabilità, non un divieto imposto dalla sintassi del linguaggio.

Una selezione concreta di Strategy riguarda l'ordinamento: secondo la dimensione dei dati e i compromessi richiesti, il contesto può scegliere un algoritmo diverso e sostituirlo nuovamente durante l'esecuzione. Gli algoritmi devono rispettare lo stesso contratto di ordinamento; il cambio non autorizza a cambiare il risultato richiesto.

9.5 Observer: notificare i dipendenti senza conoscerne i tipi

Un oggetto `Dati` può alimentare una tabella, un grafico e un monitor. Se chiama direttamente `aggiorna()` su ciascuna vista, deve conoscerne il tipo concreto e cambiare quando una vista viene aggiunta o rimossa.

L'intento di **Observer**, detto anche *Publisher–Subscribe*, è sostituire queste dipendenze con una relazione uno-a-molti: quando il soggetto cambia stato, i dipendenti registrati vengono

notificati e aggiornati automaticamente. Il soggetto conosce il contratto di notifica, anziché ogni tipo di vista.

`Subject` conserva una collezione di `Observer` e offre `attach`, `detach` e `notify` per registrare, rimuovere e notificare gli iscritti. Il contratto `Observer` dichiara `update`.

La collaborazione parte da `ConcreteSubject`, che conserva e modifica lo stato di interesse, per esempio con `setState`. Solo quando il subject è coerente dopo la modifica, `notify` percorre gli iscritti e invoca `update`. Il `ConcreteObserver` mantiene il riferimento al subject e lo stato locale da rendere consistente con esso.

Nel diagramma UML l'asterisco esprime la molteplicità di questa relazione: a un Subject possono essere associati zero o più Observer.

Nel modello **pull**, la notifica comunica soltanto che è avvenuto un cambiamento e l'Observer interroga il Subject con, per esempio, `getState`. È flessibile perché ogni osservatore legge solo i dati necessari, ma richiede il riferimento al Subject.

Nel modello **push**, `update` riceve direttamente i dati modificati: l'Observer non interroga il Subject, ma il Subject deve sapere quali dati servono agli osservatori. Il modello pull lascia maggiore libertà a osservatori con esigenze diverse; la scelta fra i due modelli dipende anche dal costo delle letture e dal contenuto della notifica.

Il disaccoppiamento permette di aggiungere osservatori senza modificare Subject. Questo vantaggio non elimina però i costi: la notifica è un broadcast, può provocare aggiornamenti inattesi e molte operazioni.

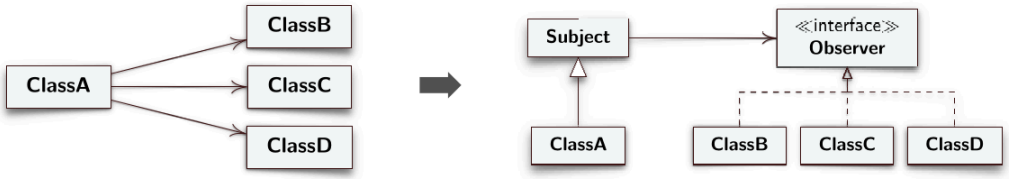


Figura 9.5: Le dipendenze dirette da ciascun osservatore vengono sostituite dal contratto Observer.

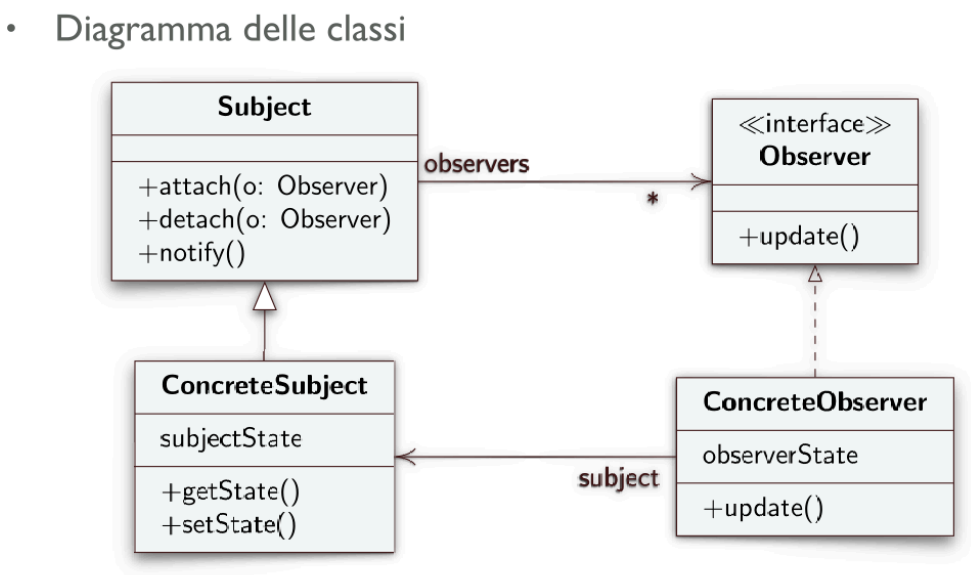


Figura 9.6: Il Subject conserva gli Observer e il ConcreteObserver legge lo stato di interesse.

Nella variante pull la notifica può indicare il completamento di un'operazione senza specificare quali campi siano cambiati. Per limitare lavoro inutile gli iscritti possono dichiarare gli eventi di interesse. Un osservatore di più subject deve ricevere o poter ricavare l'identità del mittente, per esempio con un parametro Subject di `update`. Con molti subject e pochi observer, una tabella associativa condivisa può sostituire alcune liste distribuite: è un'opzione di implementazione da valutare sul caso concreto.

Notifiche e consistenza dello stato

Le notifiche automatiche rendono difficile dimenticare l'aggiornamento ma possono ripetersi inutilmente durante una modifica composta.

Le notifiche esplicite permettono di raggruppare gli aggiornamenti; in questo caso è il chiamante a dover inviare le notifiche. Sia con notifiche automatiche sia con notifiche esplicite, lo stato pubblicato deve essere consistente.

Se un Observer modifica di nuovo il Subject, possono nascere ulteriori notifiche e cicli: il protocollo deve evitare aggiornamenti ricorsivi indesiderati.

Rimuovere le registrazioni quando non servono evita aggiornamenti superflui e riferimenti che prolungano la vita degli oggetti. In Java il problema non è normalmente un puntatore a memoria già liberata: sono invece i riferimenti e le iscrizioni non più utili che rimangono attivi.

La dismissione del Subject richiede di considerare anche i riferimenti conservati dagli Observer. Prima di cessare il servizio, il soggetto può inviare un avviso previsto dal contratto: gli osservatori smettono di interrogarlo e cancellano il riferimento che mantengono a esso. Il Subject può quindi rimuovere le proprie registrazioni. Il protocollo completa così la separazione nei due versi, evitando che restino collegamenti verso un soggetto non più utilizzabile.

L'avviso va previsto esplicitamente, per esempio come evento di chiusura: il normale `update` non significa automaticamente che il soggetto stia per scomparire. In Java questa è una

dismissione logica decisa dall'applicazione; non si deve attendere che il garbage collector esegua tale protocollo al suo posto.

9.5.1 Protocollo di aggiornamento coerente

Per un aggiornamento Observer coerente:

1. modificare completamente lo stato del ConcreteSubject;
2. decidere se notificare subito o raggruppare le modifiche;
3. chiamare `notify` solo quando lo stato è consistente;
4. per ogni Observer registrato, invocare `update`;
5. nel modello pull, fare leggere al ConcreteObserver i dati necessari dal Subject.

9.5.2 Rubrica osservabile e salvataggio su file

La rubrica è ConcreteSubject, il salvataggio è ConcreteObserver e `Dialog` configura il collegamento. Il contratto usa il mittente e un dato generico:

```
import java.util.*;
import java.io.*;

interface Observer { void update(Subject s, Object stato); }
class Subject {
    private final List<Observer> obs = new ArrayList<>();
    private boolean changed;
    public void attach(Observer o) { obs.add(o); }
    public void detach(Observer o) { obs.remove(o); }
    protected void setChanged() { changed = true; }
    public void notify(Object stato) {
        if (!changed) return;
        for (Observer o : obs) o.update(this, stato);
        changed = false;
    }
}
record Persona(String nome, String cognome, String telefono, String citta) { }
class AddrBook extends Subject {
    private final List<Persona> nomi = new ArrayList<>();
    public void insert(Persona p) {
        if (nomi.contains(p)) return;
        nomi.add(p);
        setChanged();
        notify(nomi);
    }
    public Persona find(String cognome) {
        for (Persona p : nomi)
            if (p.cognome().equals(cognome)) return p;
        System.out.println("AddrBook.find: NOT found");
        return null;
    }
}
class Store implements Observer {
```

```
    @SuppressWarnings("unchecked")
    public void update(Subject s, Object stato) {
        List<Persona> persone = (List<Persona>) stato;
        try (FileWriter f = new FileWriter("nomi.txt")) {
            for (Persona p : persone)
                f.write(p.nome() + "\t" + p.cognome()
                    + "\t" + p.telefono() + "\n");
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}
```

Il tipo Persona è qui un record di contatto, separato dagli esempi anagrafici precedenti; gli accessori sono quelli del record. Il client esegue `book.attach(store)` e inserisce `new Persona("Oliver","Stone","012345","NY")`. L'inserimento cambia prima la lista, imposta il flag, notifica e fa scrivere nel file nome, cognome e telefono separati da tabulazioni. Un duplicato secondo `equals` non genera notifica. `find("Stone")` restituisce il contatto; una ricerca assente stampa il messaggio e restituisce `null`.

`notify(Object)` è un metodo del Subject didattico, con parametro, distinto da `Object.notify()` senza parametri. Ogni osservatore viene chiamato sincronicamente e un errore di scrittura deve essere reso visibile al livello che può gestirlo.

Per eseguire il cast dal dato generico occorre sapere quale contenuto è previsto nella notifica; un contratto parametrico può evitare questo controllo a runtime.

Il flag torna falso dopo il completamento della notifica; un'eccezione prima della fine interrompe il protocollo e richiede una politica esplicita. Non si deve presumere che la notifica sia una transazione.

9.5.3 Observable storico e Reactive Streams

Nell'API storica `Observable` è una classe concreta estendibile, non un'interfaccia; estenderla occupa l'unica superclasse disponibile. Il metodo `notify()` di `Object` ha un significato di sincronizzazione tra thread e non va confuso con la notifica del pattern: l'API degli osservatori usa `notifyObservers()`. Occorre prima impostare il flag con `setChanged()`; dopo la notifica il flag è cancellato. L'Observer riceve `update(Observable,Object)`, con il mittente e un eventuale dato aggiuntivo. L'API è deprecata da Java 9 ed è mantenuta per retrocompatibilità.

Sincronia e consumatori lenti

La consegna storica è sincrona e bloccante: la chiamata attende il ritorno di `update` degli osservatori. Un consumatore lento rallenta così il produttore; separare le esecuzioni richiede anche di controllare quanto lavoro il consumatore sia pronto a ricevere.

I Reactive Streams introducono il controllo della domanda, detto **backpressure**. Le interfacce Java sono annidate in `java.util.concurrent.Flow`:

`Flow.Publisher<T>` riceve iscrizioni con `subscribe(...)`;

`Flow.Subscriber<T>` riceve `onSubscribe`, `onNext`, `onError` e `onComplete`;

`Flow.Subscription` collega le parti: `request(n)` aumenta la domanda e `cancel()` termina l'iscrizione.

Il Subscriber può richiedere un elemento, elaborarlo in `onNext` e richiedere il successivo, oppure richiedere un lotto. La domanda riguarda il numero di elementi consegnabili, non la creazione di thread.

`SubmissionPublisher<T>` offre consegna asincrona usando un esecutore; normalmente usa un pool condiviso, non un thread nuovo per ogni messaggio. Conserva un buffer per ciascun subscriber.

Se manca spazio, `submit()` può bloccare il produttore; la consegna asincrona non significa assenza assoluta di blocchi.

L'obiettivo è controllare lo squilibrio tra produttore e consumatore e separare le rispettive esecuzioni. Questo protocollo di sottoscrizione asincrona è diverso dalle pipeline di `java.util.stream`, che trasformano una sequenza e vengono attivate da un'operazione terminale.

Dalla lista notificata agli elementi pubblicati

Nella variante asincrona AddrBook contiene un `SubmissionPublisher<Persona>`; Store implementa `Subscriber<Persona>`. L'inserimento pubblica la nuova persona, invece di consegnare a ogni aggiornamento l'intera lista:

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.Flow.*;

class AddrBook implements AutoCloseable {
    private final List<Persona> nomi = new ArrayList<>();
    private final SubmissionPublisher<Persona> publ =
        new SubmissionPublisher<>();
    public void attach(Subscriber<Persona> s) { publ.subscribe(s); }
    public boolean insert(Persona p) {
        if (nomi.contains(p)) return false;
        nomi.add(p); publ.submit(p);
        return true;
    }
    public void close() { publ.close(); }
}
class Store implements Subscriber<Persona> {
    private Subscription sub;
    private final CountDownLatch fine = new CountDownLatch(1);
    public void onSubscribe(Subscription s) {
        sub = s; sub.request(1);
    }
    public void onNext(Persona p) {
```

```
            System.out.println("Store onNext: " + p.nome() + "\t" + p.cognome());
            sub.request(1);
        }
        public void onError(Throwable t) {
            System.err.println("In Store: errore " + t);
            fine.countDown();
        }
        public void onComplete() {
            System.out.println("In Store: completato");
            fine.countDown();
        }
        public void attendi() throws InterruptedException { fine.await(); }
}
```

Si riusa il record Persona del contatto. Il client collega Store, inserisce i contatti e chiude il publisher. Se deve osservare tutte le stampe, prima di terminare chiama `store.attendi()`: chiudere avvia il completamento, mentre il contatore di attesa permette di aspettare che il ciclo di vita dell'esempio sia concluso.

Il controllo della domanda riguarda invece le callback: `onSubscribe` richiede il primo elemento; ogni `onNext` completa l'elaborazione ricevuta e solo dopo richiede l'elemento successivo. Le callback di errore e completamento sono terminali. Attendere la fine e richiedere nuovi elementi hanno quindi responsabilità diverse.

Questa versione di Store stampa i singoli eventi; non equivale automaticamente al salvataggio dell'intera rubrica della versione sincrona.

Lo stesso passaggio si applica a un `NoteProducer`: il tipo pubblicato diventa `String`, mentre `SlowReceiver` e `Display` sono due Subscriber distinti configurati dal Main. Il consumatore lento esprime la propria domanda senza imporre al display la stessa velocità. Il cambiamento essenziale nel diagramma consiste nel sostituire Subject/Observer con Publisher/Subscriber, mantenendo separati produttore e consumatori; non la creazione obbligatoria di un thread per messaggio.

9.6 MVC: dati, presentazione e gestione degli input

MVC separa le responsabilità di un'applicazione interattiva in `Model`, `View` e `Controller`, affinché dati e logica, presentazione e input possano evolvere e riusarsi indipendentemente.

Il **Model** incapsula dati e logica di business, cioè le operazioni significative nel dominio del problema; non deve sapere come i dati sono mostrati né quale dispositivo genera l'input.

La **View** mostra i dati all'utente.

Il **Controller** riceve eventi di mouse o tastiera, li traduce in richieste di servizio per il Model oppure in comandi di interfaccia per la View.

Nel MVC classico, il Model svolge il ruolo di Subject; View e Controller sono ConcreteObserver. Il Model registra e notifica entrambi attraverso il contratto, senza conoscerne i tipi concreti.

La gestione dell'input segue un'altra collaborazione. La View inizializza il proprio Controller e gli delega l'evento fisico con `handleEvent`. Il Controller conserva un riferimento alla View per le azioni puramente grafiche, come evidenziare un campo o mostrare un dialogo, che non appartengono al Model. Nella collaborazione classica qui descritta, ogni View è associata al proprio Controller e viceversa.

Il Controller può osservare il Model perché una variazione dei dati può rendere leciti o illeciti alcuni input, per esempio abilitando un pulsante o attivando la sola lettura. Una View statica, come una schermata about o una pagina di errore priva di dati dinamici, può invece svolgere il proprio ruolo senza registrarsi al Model.

Nel web, una configurazione MVC distribuisce Model e Controller sul server e la presentazione nel browser. Il Controller riceve una richiesta HTTP, usa il Model e seleziona o genera una pagina HTML da mostrare. A differenza delle viste desktop che rimangono attive e ricevono notifiche, una pagina prodotta in risposta alla richiesta non richiede necessariamente la registrazione Observer. Anche se i ruoli sono distribuiti fisicamente in modo diverso, restano separati sul piano logico.

Un'applicazione Java realizzata con Spring Boot può adottare questa organizzazione web MVC. Il framework fornisce il contesto applicativo, mentre il progetto deve mantenere distinte logica del dominio, presentazione e gestione delle richieste.

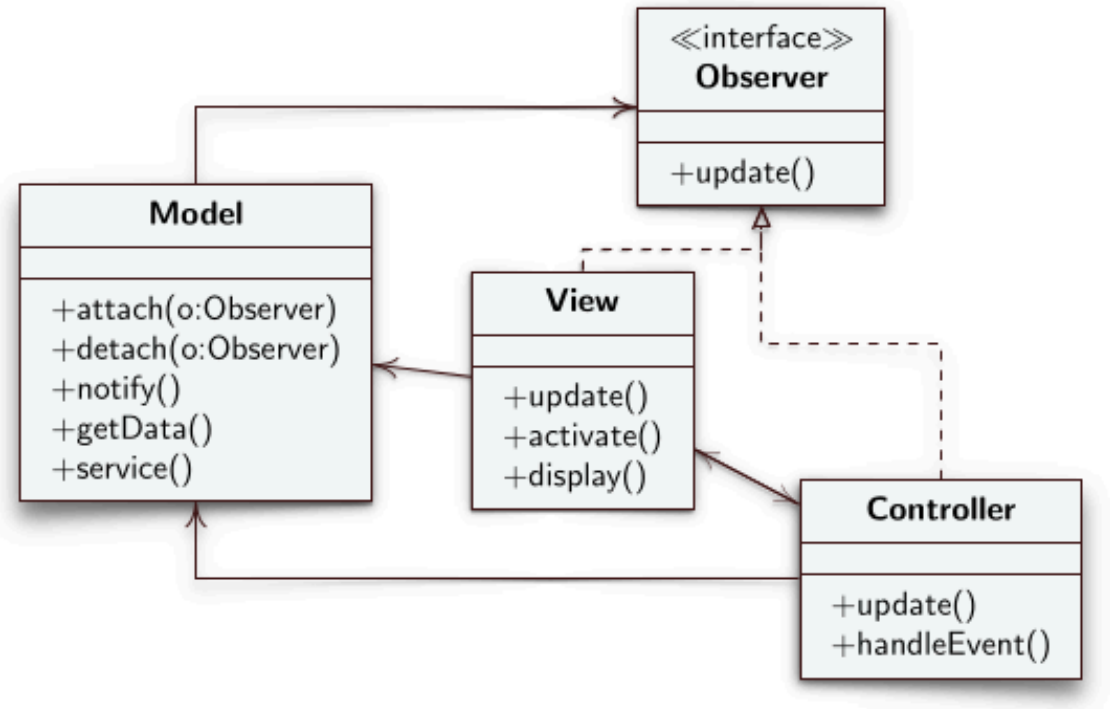


Figura 9.7: Model come Subject e View e Controller come osservatori.

Il Model può riunire Subject e ConcreteSubject, contenendo sia le registrazioni sia i dati. La separazione logica dei ruoli può essere mantenuta anche quando alcune classi sono accorpate; ciò che conta è che il Model non debba conoscere i dettagli delle interfacce utente.

La collaborazione desktop può essere ricostruita con questa sequenza:

1. la View riceve l'evento dell'utente e chiama `handleEvent()` sul Controller;
2. il Controller richiede `service()` al Model;
3. il Model aggiorna i dati e chiama la propria operazione di notifica;
4. gli Observer registrati ricevono `update()`;
5. la View legge `getData()` e chiama `display()`;
6. per un'azione puramente grafica, il Controller può chiamare `activate()` sulla View senza modificare il Model.

Questi nomi appartengono all'esempio: il criterio architetturale è separare i ruoli, non imporre nomi universali ai metodi.

9.7 Chain of Responsibility: trovare un gestore lungo la catena

Chain of Responsibility separa il mandante di una richiesta dal suo ricevente: collega più possibili gestori e fa avanzare la richiesta finché uno la tratta.

Il client invia la richiesta al primo handler. `Handler` definisce il contratto e il riferimento al successore; ciascun `ConcreteHandler` decide se la richiesta è di propria competenza, altrimenti la inoltra. Ogni gestore conosce soltanto il successore immediato, perciò la catena può essere modificata a runtime inserendo, rimuovendo o riordinando i gestori.

Nell'aiuto di una GUI, per esempio, il bottone cerca il proprio messaggio di assistenza. Se non lo possiede, inoltra la richiesta al dialogo o alla finestra che lo contiene. Il percorso può quindi coincidere con la risalita di una gerarchia Composite.

Non è però garantito che uno dei gestori tratti la richiesta e una catena lunga aumenta attraversamento e difficoltà di debugging.

Alternative del contratto Handler

Sono possibili due contratti. Con un metodo per tipo di richiesta, come `handleHelp()` e `handlePrint()`, il compilatore controlla le richieste valide, ma aggiungerne una modifica Handler e tutti i ConcreteHandler.

Con un solo `handleRequest(code)`, il *codice* identifica il tipo a runtime e l'interfaccia non cambia; si perde però sicurezza a compile-time; un codice sconosciuto può arrivare in fondo alla catena senza che venga segnalato.

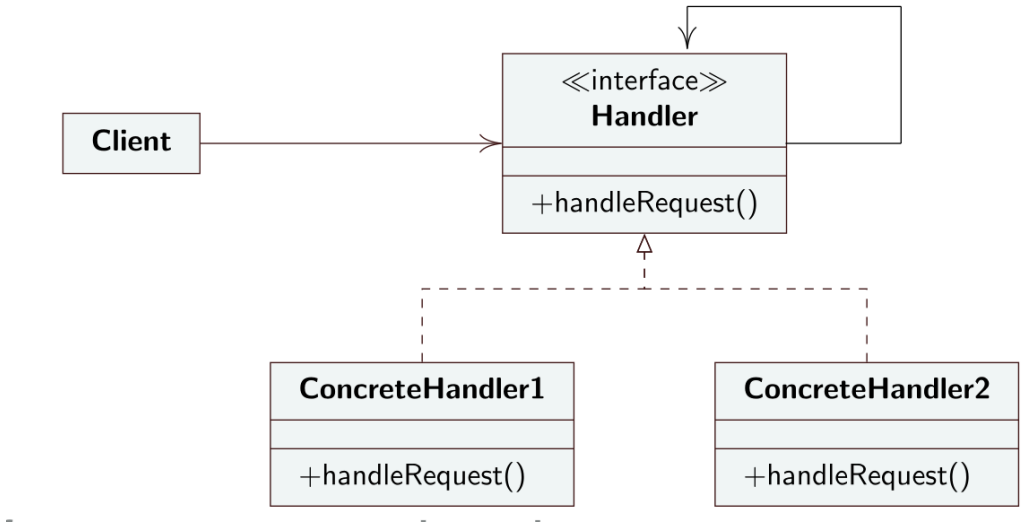


Figura 9.8: Il riferimento al successore collega i gestori senza una lista globale nel singolo handler.

Il successore può essere definito in Handler con un inoltro predefinito, oppure separatamente nei ConcreteHandler: la seconda scelta concede più controllo ma può duplicare codice. Un riferimento al padre già esistente in Composite può realizzare la risalita della richiesta di help, senza costruire una seconda rete di collegamenti.

Precisazione

La correttezza dei tipi non garantisce che un handler soddisfi la richiesta. In entrambe le varianti occorre stabilire l'esito quando il successore è assente e nessuno risponde. Il contratto con codici numerici richiede inoltre un accordo sulla codifica e controlli a runtime.

9.7.1 Esempio: aiuto contestuale

Nel sistema di aiuto, `Button` e `Text` sono ConcreteHandler: il bottone inoltra la richiesta fino al testo, che fornisce assistenza se non esiste un successore.

Aiuto contestuale: contratto e chiamate

Il contratto `Handler` dichiara `handleRequest()`, `activate()` e `setNext(Handler)`. L'ultimo metodo configura la catena; `activate` descrive il controllo, non inoltra automaticamente una richiesta di aiuto.

```
interface Handler {
    void handleRequest(); void activate(); void setNext(Handler next);
}
class Text implements Handler {
    private Handler next;
    Text(Handler h) { next = h; }
    public void handleRequest() {
        if (next != null) next.handleRequest();
        else System.out.println("Text: assistenza");
    }
```

```
    public void activate() {
        System.out.println("sono un frammento di testo");
    }
    public void setNext(Handler h) { next = h; }
}
```

Il client costruisce `Text t=new Text(null)` e un Button con successore `t`, attiva entrambi e invia due richieste al bottone. Un Button che non fornisce aiuto proprio inoltra entrambe a Text: il messaggio di assistenza viene quindi prodotto due volte. Il numero di richieste non cambia il collegamento. Il comportamento specifico di Button è una scelta del suo contratto; la catena resta valida anche aggiungendo un gestore intermedio.

9.8 Command: una richiesta diventa un oggetto

Command rappresenta una richiesta mediante un oggetto. Il contratto `Command` dichiara `execute`; un `ConcreteCommand` collega l'azione a un `Receiver` e gli delega il lavoro.

Il Client crea e collega gli oggetti. In seguito, l'`Invoker` usa il Command che conserva e ne invoca `execute`: non deve conoscere né il comando concreto né il Receiver. Configurare la richiesta ed eseguirla sono così responsabilità separate.

Essendo un oggetto, la richiesta può essere passata, salvata, inserita in una coda o in un log e composta in un `MacroCommand`, applicando Composite. Per un comando semplice, la logica può stare direttamente nel comando, senza un Receiver separato.

Stato necessario per annullare un comando

Per `undo`, un comando reversibile conserva lo stato necessario a tornare alla situazione precedente e il Receiver deve offrire l'operazione inversa.

Per più annullamenti l'Invoker mantiene uno *stack*, cioè una pila LIFO (*Last In, First Out*): l'ultimo comando eseguito è il primo annullato.

Per `redo` serve un secondo stack. Riutilizzare una stessa istanza sovrascrivendone i dati perderebbe invece la cronologia.

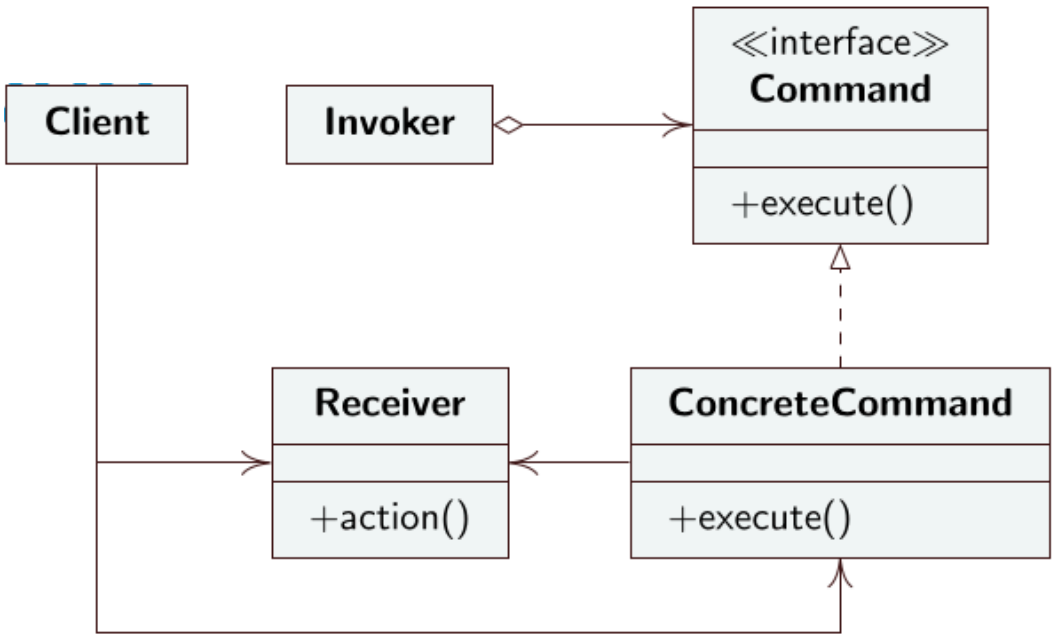


Figura 9.9: Client configura comando e ricevente; Invoker conosce soltanto Command.

I dati dell'operazione vengono forniti al ConcreteCommand quando è configurato: l'Invoker chiama poi `execute()` senza passarli nuovamente. Invoker può essere un bottone, un gestore di input o uno scheduler; non è proprietario esclusivo del comando nel modello illustrato. Nell'editor di testo, scrivere una parola, applicare il grassetto e cancellare con un `DeleteCommand` sono richieste rappresentabili con oggetti diversi. Comandi come `apriFile` e `salvaFile` possono essere riuniti in una macro.

Per annullare un comando si può eseguire un'operazione inversa oppure ripristinare uno stato precedente salvato.

Non tutte le operazioni sono reversibili. Una MacroCommand esegue una sequenza di Command, svolgendo Composite; per annullarla si procede normalmente in ordine inverso. Un comando di aggiornamento del calendario conserva timestamps in uno stack e usa `putIfAbsent` per evitare sovrascritture: se l'inserimento non è avvenuto perché la chiave esisteva, l'undo non deve cancellare una voce preesistente che quel comando non ha creato. Occorre registrare quale effetto ha realmente prodotto l'esecuzione.

9.8.1 Protocollo undo e redo a più livelli

Per l'annullamento a più livelli servono due pile: S_u per gli undo e S_r per i redo. Dopo l'esecuzione riuscita di un comando reversibile c, si inserisce c in S_u e si svuota S_r.

L'operazione `undo` rimuove il comando in cima a S_u, ne invoca l'inversa e lo trasferisce in S_r. L'operazione `redo` compie il percorso opposto: prende il comando in cima a S_r, lo riesegue e lo reinserisce in S_u.

Il simbolo c indica una singola istanza di comando; la "cima" è l'ultimo elemento inserito nella pila. La cronologia conserva dunque le istanze delle esecuzioni da annullare o ripetere.

9.8.2 Esempio: menu e calendario

Nel calendario, `Menu` è Invoker, `Calendario` è Receiver e `Updater` è ConcreteCommand: aggiungere o togliere note è un oggetto comando, quindi può offrire `unexecute`.

Effetti reversibili nel calendario e nell'editor

Nel calendario `aggiungi` associa una nota a un timestamp, `rimuovi` la elimina e `stampa` osserva gli appuntamenti. Menu riceve un Command e lo attiva; Updater conserva calendario, dati dell'operazione e memoria dell'esito. Per un inserimento con `putIfAbsent`, la voce va aggiunta alla cronologia soltanto se la chiave era libera. Il protocollo essenziale è:

```
// Corpo di execute: receiver e dati sono gia' configurati.
boolean inserito = calendario.aggiungi(timestamp, nota);
if (inserito) inserimenti.push(timestamp);

// Corpo di unexecute.
if (!inserimenti.isEmpty())
    calendario.rimuovi(inserimenti.pop());
```

La pila conserva l'ordine effettivo degli inserimenti riusciti. Se una chiave esisteva già, annullare il tentativo fallito non deve eliminare la nota precedente.

Per annullare anche modifiche o cancellazioni occorre salvare il valore precedente, non soltanto il timestamp. La separazione dei ruoli resta la stessa: Menu non conosce la mappa, Updater interpreta la richiesta e Calendario esegue l'aggiornamento.

Nell'editor, un comando di inserimento conserva testo e ricevente. Se aggiunge una stringa `s` in fondo al documento, l'inversa può cancellare gli ultimi `s.length()` caratteri, purché nel frattempo non siano intervenute modifiche incompatibili e gli undo rispettino l'ordine inverso. Un inserimento in una posizione arbitraria richiede anche la posizione; cancellare gli ultimi caratteri non è allora un'inversa generale. Il comando di salvataggio può controllare il flag "documento modificato": salva quando è vero e non riscrive inutilmente quando è falso.

Nel diagramma delle classi di Command, l'Invoker conserva stabilmente il riferimento al Command; il Client può usarlo solo durante il wiring. L'assenza della freccia temporanea Client–Invoker in uno schema semplificato non cancella il messaggio di configurazione visibile nel diagramma di sequenza.

9.9 Mediator: incapsulare il protocollo cooperativo

In una finestra di ricerca, i controlli `Category`, `SubCategory`, `Search`, `Ok` e `Cancel` devono reagire gli uni ai cambiamenti degli altri. Se si chiamano tutti reciprocamente, la logica di coordinamento si distribuisce fra molti collegamenti.

Mediator raccoglie il protocollo di interazione in un oggetto. Un `Colleague` conosce il Mediatore e gli comunica l'evento, senza conoscere gli altri Colleague. `Mediator` è il contratto di comunicazione; `ConcreteMediator` conserva i riferimenti necessari e decide come coordinare i ConcreteColleague.

Nell'esempio i controlli notificano il mediatore, che sceglie quali aggiornare. Le relazioni molti-a-molti vengono così sostituite da relazioni uno-a-molti centrate sul mediatore.

Dipendenze e costo del coordinamento

La dipendenza comportamentale non scompare: se cambia una categoria, una sottocategoria deve ancora aggiornarsi. Si elimina la dipendenza strutturale diretta fra colleghi e si concentra nel mediatore la logica con cui collaborano.

I Colleague diventano più semplici e riusabili; il compromesso è un ConcreteMediator potenzialmente complesso e specifico dell'applicazione.

Mediator non è Facade: Facade semplifica l'accesso unidirezionale a un sottosistema che non conosce la facciata; i Colleague conoscono invece il Mediator e comunicano con esso in entrambe le direzioni. Mediator e Observer possono collaborare quando i Colleague notificano il mediatore mediante Observer.

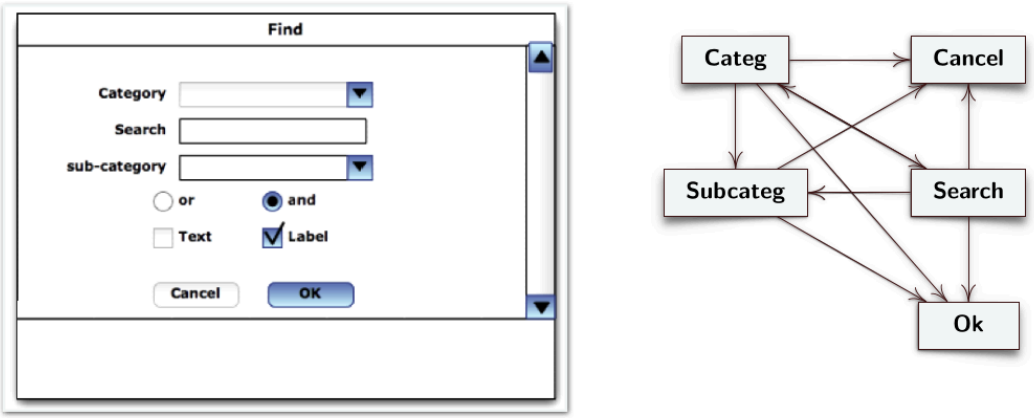


Figura 9.10: La finestra di ricerca e le dipendenze dirette fra i controlli prima del mediatore.

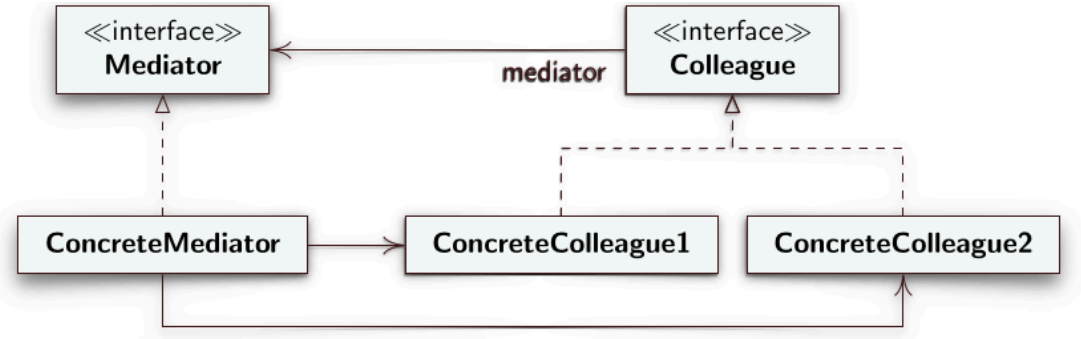


Figura 9.11: La finestra di ricerca centralizza le interazioni dei controlli in un mediatore.

Quando Category cambia, il mediatore aggiorna SubCategory; se Search non è valido può disabilitare Ok. I colleghi non scelgono direttamente quali altri controlli aggiornare. ConcreteMediator deve invece conoscere le operazioni concrete degli oggetti che partecipano alla collaborazione: la complessità e il costo delle estensioni sono concentrati in quel punto.

Il vantaggio è poter riusare i controlli e leggere la logica di cooperazione in un unico luogo.

Se Mediator e ConcreteMediator coincidono, si riducono le classi ma si perde l'interfaccia che permetteva ai colleghi di usare implementazioni alternative senza modifiche. Il mediatore è di solito specifico dell'applicazione. Può ricevere notifiche Observer dai colleghi, ma stabilisce poi il coordinamento: Observer distribuisce eventi, Mediator incapsula il protocollo cooperativo.

Un ciclo di dipendenze o un disegno fitto di frecce può motivare una revisione, ma la disposizione grafica delle linee non è una metrica affidabile della bontà del progetto.

9.9.1 Coordinare lettura, numeri e nomi

Un secondo esempio impiega `Central` come Mediator e quattro ConcreteColleague: `KeyListener`, `Numeri`, `Sommatore` e `Nomi`, collegati attraverso il contratto `Elemento`. KeyListener legge un dato dalla tastiera e lo restituisce a Central; il mediatore distingue numeri e stringhe e chiama `put` sui partecipanti interessati. Numeri, Sommatore e Nomi non devono perciò conoscere reciprocamente le proprie concrete.

La sequenza inizia con Central che invoca `doWork()` su KeyListener. Ottenuto il dato, distribuisce richieste `put()` e le eventuali segnalazioni `alert()` previste dal protocollo. Se un collega riconosce la condizione di arresto, chiama `stop()` sul mediatore, che avvisa gli altri colleghi. Nel caso in cui sia Numeri a riconoscerla, la comunicazione torna quindi da Numeri a Central e da Central agli altri; Numeri non li arresta direttamente. La parola o condizione concreta di arresto e le regole esatte delle elaborazioni devono essere specificate dal dominio: il pattern determina dove coordinarle, senza fissarne il contenuto.

9.10 Iterator: attraversare senza esporre la rappresentazione

Iterator permette di attraversare una collezione senza esporne la rappresentazione.

Il client chiede l'iteratore all'aggregato: `Aggregate` dichiara `createIterator`, mentre `ConcreteAggregate` contiene gli elementi effettivi.

Il percorso usa poi il contratto `Iterator`, con operazioni come `hasNext()` e `next()`. Il `ConcreteIterator` conserva la posizione corrente, per esempio un indice, e sa come avanzare nella struttura specifica. Il client può così attraversarla senza distinguere array, lista o albero. Il ciclo `for-each` di Java applica questo principio agli oggetti Iterable.

9.11 Null Object: rappresentare un comportamento neutro

Null Object sostituisce l'assenza di un collaboratore con un oggetto che offre un comportamento neutro; può essere introdotto anche mediante refactoring. Invece di restituire `null` e ripetere `if (x != null)`, si fornisce un `NullObject` che implementa lo stesso contratto di `RealObject` ma esegue un'azione neutra. Riduce i controlli e previene una `NullPointerException`, cioè l'errore prodotto tentando di invocare un membro su un riferimento nullo.

L'eccezione importante è diagnostica: il comportamento neutro può nascondere il fatto che un oggetto sia assente in modo inatteso e rendere il difetto più difficile da individuare. Va quindi usato solo quando "non fare nulla" è semanticamente corretto.

9.12 Isolare la causa del cambiamento

Un procedimento di scelta è: identificare se varia un passo di un algoritmo (Template Method), l'algoritmo intero (Strategy), lo stato interno (State), il numero di dipendenti (Observer), il destinatario di una richiesta (Chain), il momento di esecuzione della richiesta (Command), la

rete di comunicazioni (Mediator) o la rappresentazione di un attraversamento (Iterator). Poi definire il contratto minimo e verificare il compromesso: classi aggiuntive, notifiche inattese, richiesta non gestita, stato necessario per undo, oppure complessità concentrata nel mediatore.

La scelta di un pattern parte dalla causa del cambiamento: quale dipendenza rende difficile estendere o modificare il sistema? Un contratto utile isola quella dipendenza e lascia stabile il codice che non deve conoscerla.

Tabella 9.1: Dipendenze da contenere e pattern adatti a isolarle.

Dipendenza o problema	Decisione di progetto	Pattern o tecniche
Istanziazione con una classe concreta	Centralizzare la scelta o clonare un prototipo attraverso un contratto.	Factory Method, Prototype.
Operazione o destinatario specifico della richiesta	Separare chi richiede da chi gestisce e da quando la richiesta viene eseguita.	Chain of Responsibility, Command.
Piattaforma hardware o software	Isolare il codice specifico e configurare l'implementazione.	Factory, Bridge.
Rappresentazione e implementazione interna	Nascondere struttura e costruzione dietro interfacce stabili.	Factory, Bridge, Memento, Proxy.
Algoritmo soggetto a sostituzione	Isolare una variante di algoritmo, un passo o il comportamento di uno stato.	Builder, Iterator, Strategy/State, Template Method, Visitor.
Accoppiamento stretto fra componenti	Riorganizzare creazione, accesso, richieste e notifiche.	Abstract Factory, Bridge, Chain, Command, Facade, Mediator, Observer.
Troppe estensioni tramite sottoclassi	Comporre oggetti attraverso contratti e limitare le gerarchie non necessarie.	Bridge, Chain, Composite, Decorator, Observer, Strategy.
Classi che non si possono modificare	Adattare il protocollo oppure avvolgere l'oggetto aggiungendo comportamento.	Adapter, Decorator, Visitor.

Il riuso per ereditarietà è detto a scatola bianca perché può richiedere conoscenza del funzionamento della superclasse e delle dipendenze fra i suoi punti di estensione. Il riuso per composizione è detto a scatola nera perché usa contratti pubblici e può cambiare i collaboratori a runtime. Nessuna delle due tecniche sostituisce il ragionamento sull'intento: la composizione è vantaggiosa quando le dimensioni devono variare indipendentemente, mentre una vera specializzazione può giustificare l'ereditarietà.

Per valutare una soluzione creazionale occorre: individuare se varia numero, tipo, famiglia, configurazione o copia degli oggetti; separare client e concreta attraverso il contratto necessario; centralizzare la costruzione nel ruolo appropriato; definire chi possiede e reimposta lo stato; infine verificare con test che oggetti e dipendenze consegnati rispettino invarianti e contratti.